



UNIVERSITATEA TEHNICĂ

DIN CLUJ-NAPOCA

**FACULTATEA DE AUTOMATICĂ ȘI CALCULATOARE
DEPARTAMENTUL CALCULATOARE**

TITLUL LUCRĂRII DE DISERTAȚIE

LUCRARE DE DISERTAȚIE

Absolvent masterand: **Prenume NUME**

Coordonator științific: **titlul științific Prenume NUME**

Iulie, 2026



UNIVERSITATEA TEHNICĂ
DIN CLUJ-NAPOCA

FACULTATEA DE AUTOMATICĂ ȘI CALCULATOARE
DEPARTAMENTUL CALCULATOARE

DECAN
Prof.Dr.Ing. Vlad MUREȘAN

DIRECTOR DEPARTAMENT
Prof.Dr.Ing. Rodica POTOLEA

Student masterand: **Prenume NUME**

TITLUL LUCRĂRII DE DISERTAȚIE

1. **Enunțul temei:** *Scurtă descriere a temei lucrării de disertație și datele inițiale*
2. **Conținutul lucrării:** *(enumerarea părților componente) Exemplu: Pagina de prezentare, aprecierile coordonatorului, titlul capitolului 1, titlul capitolului 2, ..., titlul capitolului n, bibliografie, anexe.*
3. **Locul documentării:** *Exemplu: UTCN, Cluj-Napoca*
4. **Consultanți:**
5. **Data emiterii temei:** *Exemplu: noiembrie 2025*
6. **Data predării:** *Exemplu: 5 iulie 2026*

Student masterand:

Coordonator științific:

**FACULTATEA DE AUTOMATICĂ ȘI CALCULATOARE**
DEPARTAMENTUL CALCULATOARE**Declarație pe proprie răspundere privind
autenticitatea lucrării de disertație**

Subsemnatul(a) Prenume NUME, legitimat(ă) cuserianr., CNP, autorul lucrării TITLUL LUCRĂRII DE DISERTAȚIE, elaborată în vederea susținerii examenului de finalizare a studiilor de disertație la Facultatea de Automatică și Calculatoare, Programul de studii, din cadrul Universității Tehnice din Cluj-Napoca, sesiunea (iulie/septembrie) a anului universitar 2025-2026, declar pe proprie răspundere că această lucrare este rezultatul propriei activități intelectuale, pe baza cercetărilor mele și pe baza informațiilor obținute din surse care au fost citate în textul lucrării și în bibliografie.

Declar că această lucrare nu conține porțiuni plagiate, iar sursele bibliografice au fost folosite cu respectarea legislației române și a convențiilor internaționale privind drepturile de autor.

Declar, de asemenea, că această lucrare nu a mai fost prezentată în fața unei alte comisii de examen de disertație.

În cazul constatării ulterioare a unor declarații false, voi suporta sancțiunile administrative, respectiv, *anularea examenului de disertație*.

Sunt de acord ca, pe tot parcursul vieții, în cazul în care este necesar și se va dori verificarea autenticității lucrării mele să fiu identificat și verificat în baza datelor declarate de mine.

Data

.....

Prenume NUME

.....

Semnătura

Cuprins

Capitolul 1	Introducere	1
Capitolul 2	Obiectivele cercetării	6
2.1	Conținut	6
2.2	Secțiune	7
2.2.1	Subsecțiune	7
Capitolul 3	Studiu bibliografic/Stadiul actual în domeniu	8
3.1	Distcc	8
3.2	Ccache	10
3.3	Sccache	12
3.4	DiSCo	12
Capitolul 4	Prezentarea proiectului	14
4.1	Precizări asupra conținutului și a modului de organizare	14
4.1.1	Subsecțiune	14
Capitolul 5	Rezultate teoretice și experimentale	15
Capitolul 6	Concluzii	16
6.1	Conținut	16
6.2	Detalii tehnice	16
6.2.1	Dimensiune	16
Bibliography		17
Anexa A	Pseudo-cod sau cod (dacă există)	19

Capitolul 1

Introducere

Limbajele de programare compilate, precum **C** și **C++**, sunt de cele mai multe ori alegerea corectă pentru dezvoltarea de sisteme și aplicații pentru care performanța în timpul rulării este un aspect de importanță critică, însă alegerea lor prezintă inconveniente în ceea ce privește procesul de build. Cerințele ridicate de memorie și timpii îndelungați de compilare cresc proporțional cu dimensiunea proiectului aflat în dezvoltare.

Timpii îndelungați de compilare sunt de natură să încetinească drastic procesul de dezvoltare, ceea ce determină întârzieri ce afectează livrarea produsului către clienți și pierderi costisitoare de timp pentru evoluția proiectului.

Evident, investiția în hardware mai performant pentru a reduce timpul de compilare poate diminua magnitudinea problemei, însă se estimează că în următorii zece ani, puterea de procesare se va satura (legea lui Moor nu va mai fi valabilă începând din 2036 [1]), iar accentul se va muta asupra optimizării resurselor de rețea. Progresele în domeniul comunicațiilor vor permite transferuri rapide și eficiente de date între nodurile sistemelor distribuite, susținând astfel implementarea și extinderea proceselor distribuite de compilare. Această abordare permite fragmentarea și procesarea paralelă a sarcinilor de build pe mai multe noduri din rețea, fie ele stații locale, servere dedicate sau resurse din cloud. Drept rezultat, proiectele software vor putea beneficia de timpi reduși de compilare și de o utilizare mai eficientă a resurselor disponibile, contribuind la creșterea productivității și la reducerea blocajelor din ciclurile de dezvoltare.

Compilarea folosind mai multe sisteme de calcul care comunică prin rețea este susținută și de faptul că, odată cu răspândirea utilizării generale a calculatoarelor și a resurselor cloud, timpii de neutilizare ai sistemelor (Idle Times), au crescut considerabil. În instituții educaționale, calculatoarele au o durată de neutilizare medie de 28% (adică 2672 de ore din total de 9550 de ore de funcționare) [2]. Așadar, exploatarea resurselor de calcul ale sistemelor neutilizate reprezintă o eficientizare binevenită pentru reducerea duratei de build.

Pentru a putea înțelege de ce durata procesului de compilare pentru **C/C++** (dar mai ales pentru **C++**) este semnificativ mai mare față de alte limbaje de programare, găsim esențial să explicăm modul de desfășurare al acestui proces.

Transformarea codului sursă într-un fișier executabil este compusă din trei etape majore. Primul pas îl reprezintă expandarea directivelor de preprocesare, moment în care dependențele dintre fișierele sursă ale proiectului sunt determinate. Rezultatul etapei de preprocesare îl reprezintă o colecție de unități de translație (Translation Units), fiecare fișier sursă având asociat un astfel de **translation unit**. Urmează etapa de compilare, în urma căreia unitățile de translație sunt transformate în fișiere ce conțin cod mașină, numite fișiere obiect (Object Files). În ultima etapa, numită link-editare, toate fișierele obiect rezultate în urma compilării sunt conectate pentru a genera fișierul executabil al proiectului [3].

Etapa de preprocesare este [3] responsabilă de pregătirea fișierelor sursă pentru a fi furnizate ca intrare pentru compilator. În **C/C++**, fiecare simbol poate fi definit o singură dată, însă declararea simbolului poate să apară de mai multe ori, întrucât fiecare simbol utilizat într-o unitate de translație, trebuie să aibă o declarație locală. Această chestiune este tratată de mecanismul de includere a fișierelor, folosind directiva de preprocesare **#include**. Figura 1.1 exemplifică mecanismul de preprocesare al fișierelor sursă.

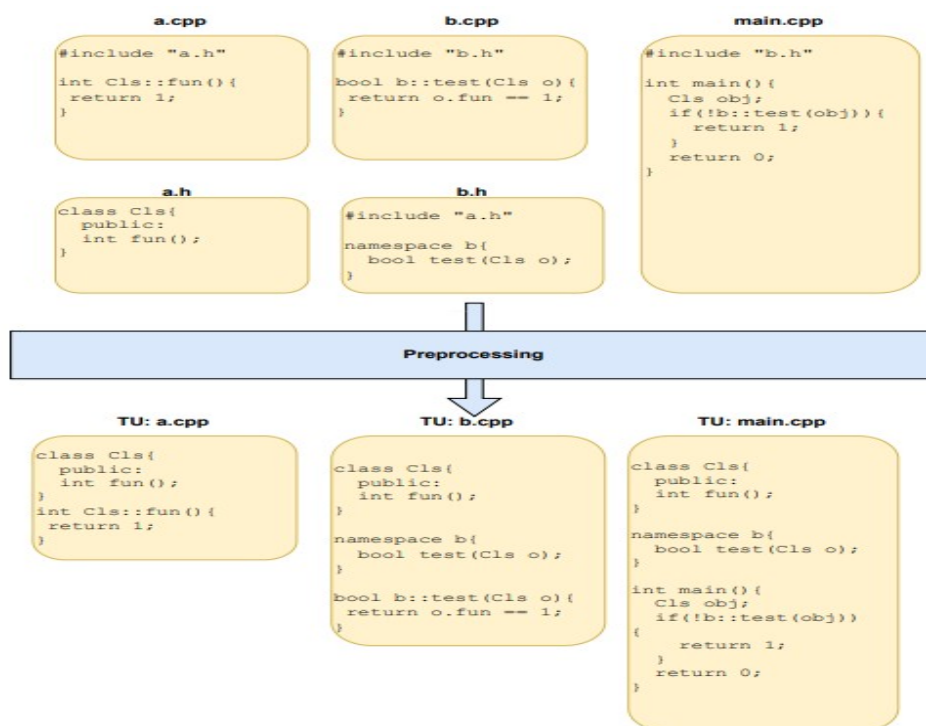


Figura 1.1: Preprocesarea fișierelor sursă [3]

Unitățile de translație sunt furnizate ca date de intrare pentru compilator, care detectează erorile de sintaxă și produce fișierele obiect, care deși conțin cod mașină, nu pot fi încă executate. Figura exemplifică etapa de compilare.

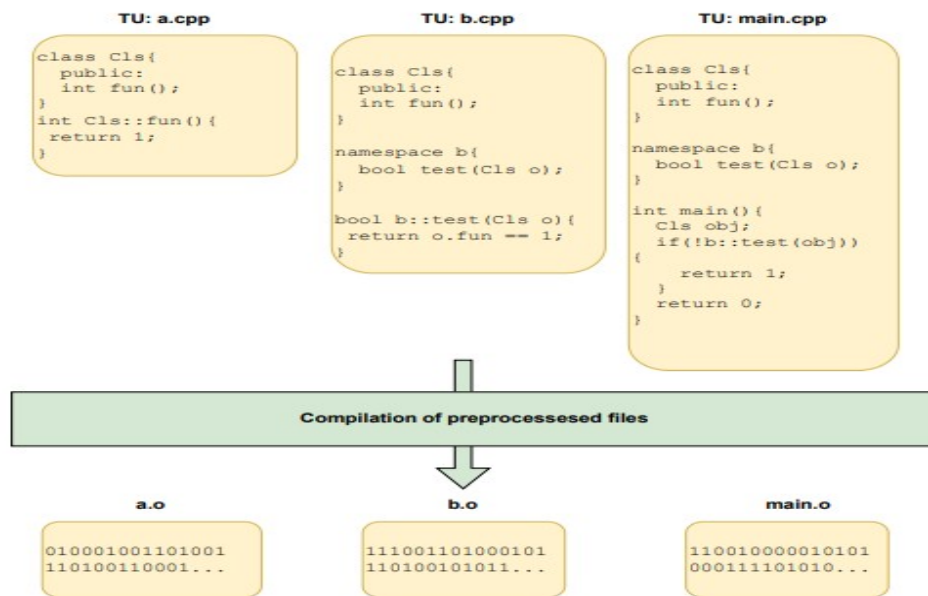


Figura 1.2: Compilarea unităților de translație [3]

Etapa de linking [3] conectează fișierele obiect pentru ca declarațiile locale să fie disponibile în celelalte unități de translație. Ieșirea linker-ului reprezintă un fișier executabil, care este rezultatul final al procesului de build.

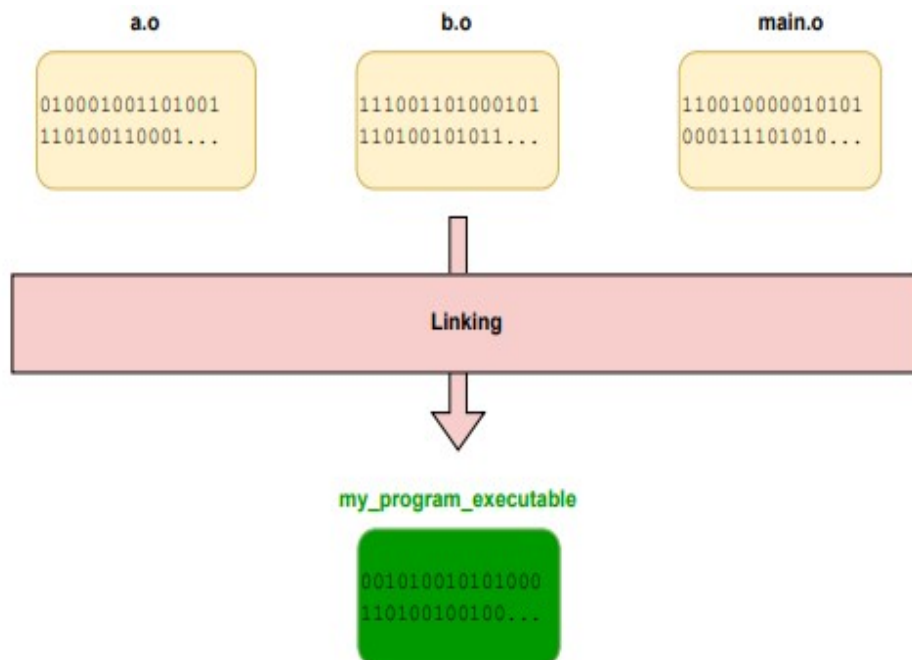


Figura 1.3: Generearea fișierului executabil [3]

Proiectele complexe, în care numărul de dependențe este mare, suferă des de recompilarea unui număr mare de fișiere, ceea ce crește semnificativ durata de compilare. Motivul pentru care compilarea excesivă este greu de evitat îl reprezintă faptul că, dacă un fișier header care este inclus de un număr mare de fișiere sursă este modificat, atunci se declanșează recompilarea tuturor fișierelor sursă care depind de acel header. Când numărul de fișiere header incluse este mare, există șansa să se creeze cicluri de dependență, o situație în care mai multe fișiere se includ reciproc [4].

Totodată, mecanismul de **templating** din **C++**, permite definirea de clase și funcții generice, care pot fi instanțiate folosind tipuri de date diverse. Întrucât **C++** este un limbaj de programare cu sistem static de tipuri (statically typed), de fiecare data când un template este invocat pentru un nou tip de argument, codul pentru implementarea specifică acelui tip de date este generat în mod automat de către compilator [5]. În cazul în care template-ul este instanțiat pentru foarte multe tipuri de date diferite, compiler-ul generează un volum semnificativ de cod repetitiv, ceea ce poate încetini procesul de build [6].

Pentru a atrage atenția asupra scenariilor în care timpul de build devine o problemă, voi invoca creșterea exponențială a volumului de cod sursă care intră în componența unor proiecte open-source cunoscute, precum **Linux Kernel** și **Android**[7]. Dimensiunea codebase-ului pentru acestea se ridică la ordinul zecilor de milioane de linii de cod. Creșterea volumului de cod de-a lungul timpului este vizibilă în Figura 1.4.

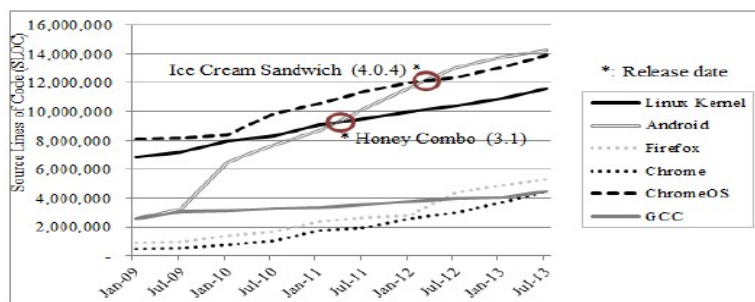


Figura 1.4: Creșterea codebase-ului de-a lungul anilor pentru câteva proiecte open-source [8]

Pentru sistemul de operare mobil **Android 4.2.2**, timpul de build total ajunge la 51 de minute [8].

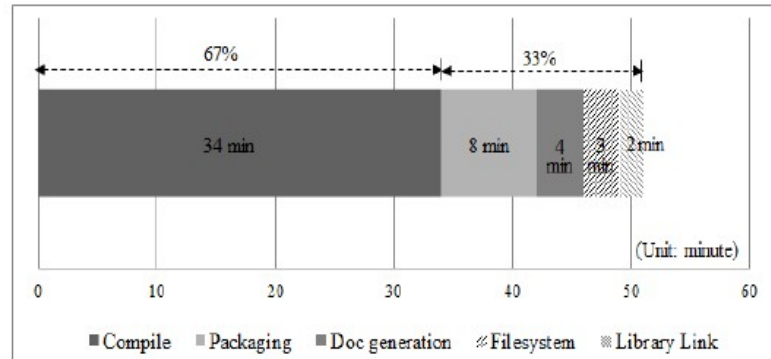


Figura 1.5: Timp de build Android [8]

Capitolul 2

Obiectivele cercetării

2.1 Conținut

Obiectivul principal al cercetării constă în utilizarea tehnologiilor moderne de comunicare din domeniul sistemelor distribuite pentru a implementa un nou sistem de compilare distribuită pentru proiecte C/C++, care să ofere performanțe competitive în ceea ce privește eficientizarea procesului de compilare, comparativ cu alternativele deja existente. Sistemul dezvoltat propune o abordare eficientă, scalabilă și flexibilă din punct de vedere arhitectural, de a crește viteza procesului de compilare, fără a introduce incompatibilități cu uneltele deja folosite în dezvoltare proiectelor.

Compilarea proiectelor dezvoltate folosind C/C++ este compusă din etape multiple de transformare a codului sursă: preprocesarea, compilarea, generarea de cod mașină pentru platforma vizată și link-editarea. Unele operațiuni aferente procesului de compilare pot fi executate independent, în paralel, însă gestiunea dependențelor dintre fișierele componente ale proiectului, artefactele generate și setările de configurare ale compilatorului introduce un grad însemnat de complexitate. Un obiectiv ce trebuie îndeplinit de către orice sistem distribuit de acest tip îl reprezintă identificarea corectă a sarcinilor care pot fi preluate de către sistemele de calcul auxiliare, spre a fi paralelizate, astfel încât să fie menținută corectitudinea și capacitatea de reproducere facilă a procesului de compilare.

Lucrarea investighează modul în care sarcinile de compilare pot să fie distribuite într-o manieră eficientă către mai multe noduri de calcul din rețea, pentru a se minimiza suprautilizarea resurselor de calcul, precum și a canalelor de comunicare. De asemenea, sistemul trebuie să țină în evidență disponibilitatea sistemelor ce contribuie la procesul de build și să își păstreze o stare de funcționare consistentă în cazul apariției erorilor în timpul transmisiei de date în rețea.

2.2 Sectiune

2.2.1 Subsectiune

Capitolul 3

Studiu bibliografic/Stadiul actual în domeniu

În cadrul acestei secțiuni, vom prezenta câteva dintre soluțiile deja existente pentru a distribui sarcina de compilare a unui proiect asupra mai multor sisteme de calcul ce comunică prin intermediul unei rețele.

3.1 Distcc

Distcc[9] oferă posibilitatea de a distribui sarcina de compilare între mai multe sisteme ce comunică în rețea, fără a fi nevoie ca sistemele participante să partajeze același sistem de fișiere, fără sincronizare de clock-uri sau să aibă acces la aceeași versiune a codului sursă.

Distcc funcționează prin distribuirea sarcinilor de lucru de la un sistem client, către nodurile server, care vor realiza compilarea. Inițial, se analizează comanda de compilare executată de către client, pentru a determina dacă sarcina poate fi distribuită, se aleg nodurile care să deservească cererea, apoi se rulează local preprocesorul asupra fișierelor ce vor fi supuse compilării. Ulterior, nodurile server primesc request-uri ce conțin fișierele sursă preprocesate și returnează către client fișierele obiect rezultate în urma compilării sau o listă cu erorile de compilare, în cazul unui eșec.

Aplicația client este invocată ca un wrapper pentru compilator-ul obișnuit (GCC de obicei), de către script-urile de build rulate de **GNU Make**. Comunicare între client-ul Distcc și nodurile server se realizează prin protocoalele de rețea **TCP** sau **SSH**, folosind un set simplu de mesaje pentru a trimite cereri de compilare: mesajul pentru cerere conține versiunea de protocol utilizată, argumentele din linia de comandă pentru invocarea compilatorului și fișierele sursă, în timp ce răspunsul unui nod server returnează fișierele obiect aferente.

Distcc oferă posibilitate ca mesajele interschimbate între nodurile client și server să fie comprimate înainte de trimitere, pentru a reduce dimensiune volumului de date tri-

mis, folosind metoda de comprimare **LZO**[10], care se concentrează asupra obținerii unei viteze cât mai bune pentru operația de decompresie. Micșorarea volumului de date folosind compresia, îmbunătățește throughput-ul de cereri transmise între participanții sistemului Distcc.

Planificarea pentru lansarea noilor sarcini de compilare folosește un mecanism simplu, prin care fiecare participant păstrează un fișier de stare, iar suprasolicitarea unui nod este împiedicată prin definirea unui număr limită de job-uri de compilare care pot fi soluționate simultan, care ia o valoare dependentă de numărul de procesoare fizice de care dispune fiecare nod. La inițializare, fiecare nod pornește numărul maxim de procese Distcc-server, care ulterior așteaptă inițierea unei conexiuni de către un client care va solicita compilarea. Pentru a putea coordona activitatea proceselor server care rulează simultan pe fiecare nod, se folosesc fișiere lock Unix, care sunt eliberate automat de către un proces în momentul terminării sale.

Un dezavantaj al sistemului distribuit Distcc îl constituie faptul că nu există niciun mecanism de estimare a timpului de compilare pentru un nod, deoarece această valoare nu este dependentă doar de dimensiunea codului sursă, ci și de aspecte precum complexitatea codului sursă sau specificațiile hardware ale nodului respectiv. Așadar, pătrunderea unui nod foarte lent în rețeaua de servere Distcc, poate încetini semnificativ procesul de build, deoarece nodul solicitant va trebui să aștepte ca fiecare nod server să își finalizeze sarcina, înainte de a primi fișierele obiect și de a invoca linker-ul.

Totodată, un inconvenient care nu trebuie neglijat este faptul că Distcc este o aplicație care rulează exclusiv pentru sisteme de operare de tip **Linux**, ceea ce nu rezolvă problema compilării distribuite pentru **Windows**.

La data prezentării inițiale a Distcc, autorul afirmă că micșorarea timpilor de build este o valoare ce se modifică liniar, odată cu numărul de core-uri de procesare implicate, cel puțin pentru un număr mic de core-uri (pentru 3 calculatoarele, build-ul a fost de 2.8 ori mai rapid). În ceea ce privește scalabilitatea sistemului Distcc, trebuie luat în calcul că dintre toate sarcinile aferente procesului de build: execuție Makefile, preprocesare, compilare, linking, doar compilarea este distribuită la mai multe noduri [11]. Pentru a determina care este limita de paralelizare a procesului de build, trebuie consultată Legea lui Amdahl, vizibilă în Figura 3.1:

$$speedup_{max} = \frac{1}{f + \frac{1-f}{P}}.$$

Figura 3.1: Legea lui Amdahl

Chiar dacă numărul de procesoare **P** tinde spre infinit, aportul de performanță este limitat de porțiunea din proces ce nu poate fi paralelizată [11].

Figura 3.2 exemplifică modul de funcționare al Distcc.

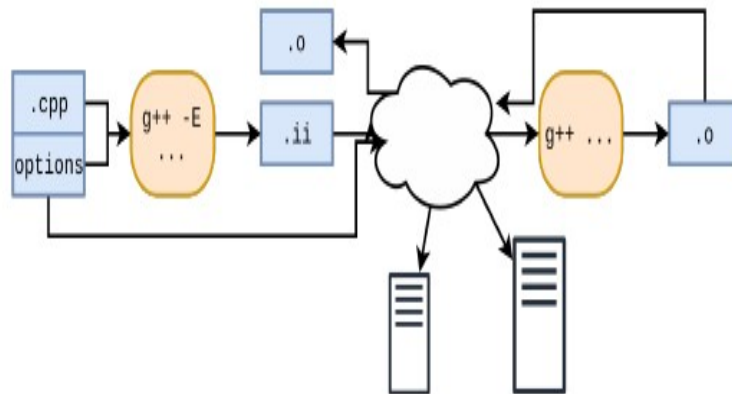


Figura 3.2: Funcționarea Distcc [12]

3.2 Ccache

Deși nu reprezintă un sistem distribuit, **Ccache**[13] este un mecanism caching pentru rezultatul compilării de fișiere sursă foarte des folosit alături de metoda descrisă anterior, Distcc, pentru a micșora și mai mult durata de build distribuit. Această combinație dintre **Distcc** și **Ccache** a fost studiată atent la Institutul CERN[12], întrucât procesul de build inițial pentru un proiect poate ajunge la 5 ore.

Sistemele tradiționale de build, precum **GNU Make** și **Ninja**, analizează timestamp-ul ultimei modificări a unui fișier sursă pentru a determina dacă compilatorul va fi invocat asupra unui fișier sursă sau nu. Problema apare în momentul schimbării de branch pentru sisteme de versionare precum **Git**, întrucât timestamp-ul fișierelor sursă își actualizează valoarea chiar dacă, conținutul nu se modifică, evenimentul fiind de natură să determine o recompilare completă a întregului proiect.

O proprietate interesantă a procesului de build pentru limbajele **C/C++** o constituie faptul că output-ul este complet determinat de conținutul surselor preprocesate și de argumentele de rulare ale compilatorului. Această proprietate este utilizată de către Ccache pentru a stoca output-ul compilării pentru un anumit fișier sursă, iar mai apoi să îl redea, în momentul începerii unui nou proces de build, evitând astfel apelarea inutilă a compilatorului pentru un fișier ce nu a fost modificat.

Sarcina principală pe care Ccache o îndeplinește o reprezintă determinarea existenței unui fișier object file asociat fișierului sursă de intrare, prin calcularea unei valori de dispersie bazată pe conținutul fișierului sursă și a flag-urilor folosite la compilarea inițială.

Ccache suportă mai multe moduri de operare, cel mai simplu dintre acestea fiind numit **preprocessor mode**, care presupune rularea preprocesorului asupra fișierului sursă

de intrare, iar mai apoi calcularea valorii hash pe baza conținutului unității de translație rezultate [14]. Dezavantajul acestui mod de funcționare este necesitatea de a apela întotdeauna preprocesorul asupra fișierului sursă de intrare, ceea ce încetinește lookup-ul.

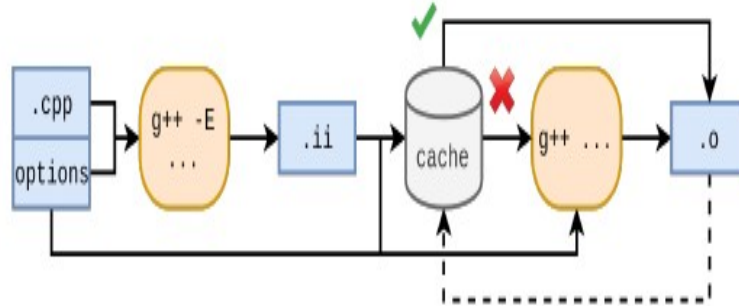


Figura 3.3: Ccache preprocessor mode [12]

De asemenea, Ccache poate funcționa într-un mod care evită pe cât posibil rularea preprocesorului, această acțiune fiind realizată doar în eventualitatea unui **cache miss**. În cadrul acestui mod de operare, valoarea hash este calculată direct pe baza fișierului sursă, fără preprocesare, iar valoarea este inclusă în cadrul unui fișier **manifest**, care conține referințe către rezultatele precedente al compilării (fișiere obiect, dependențe), care au fost produse pentru același hash, căi către fișierele header accesate la timpul compilării și valori checksum pentru fișierele incluse. În cazul unui cache miss, este necesară recalcularea valorii hash pentru fișierul de intrare și actualizarea manifest-ului.

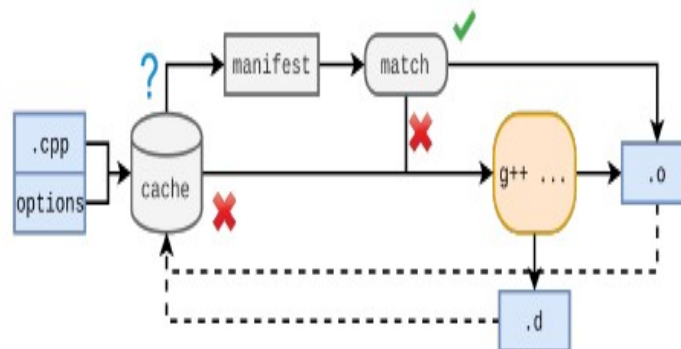


Figura 3.4: Ccache direct mode [12]

În cazul folosirii Ccache împreună cu sistemul distribuit Distcc, apare problema evidentă: cache-ul de compilare nu este distribuit pentru toate nodurile rețelei, ci fiecare nod participant are acces doar la cache-ul său local.

3.3 Sccache

Sscache[15] reprezintă varianta distribuită ca sistemului Ccache, care adaptează cache-ul inițial pentru a putea fi găzduit în cadrul unui serviciu cloud (sau pur și simplu remote) de stocare, precum: Amazon S3, Redis sau Cloudflare R2, ceea ce soluționează nevoia ca fiecare nod participant la procesul de compilare să dețină un cache local de fișiere obiect, care conține înregistrări doar pentru datele de intrare prelucrate de nodul respectiv.

Această soluție oferă un mecanism wrapper peste Ccache, astfel încât accesul la datele stocate remote să fie reglementat într-o manieră client-server [16]. Spre deosebire de Ccache, pentru calcularea unei valori hash se invocă întotdeauna preprocesorul, având astfel un comportament similar cu modul de operare **preprocessor mode**, dar se face offloading pentru volumul de date.

3.4 DiSCo

Dacă în cazul utilizării Distcc, singura dintre sarcinile din procesul de build care este paralelizată, este invocarea compilatorului asupra fișierelor preprocesate, **DiSCo** [17] propune o abordare inedită, în care preprocesarea, compilarea și link-editarea sunt efectuate în mod paralel, utilizând noduri remote.

Distcc utilizează schema de preprocesare locală, mult mai rudimentară, prin care nodul care solicită compilarea este singurul responsabil pentru realizarea etapei. Întrucât toată operațiunea are loc local, nu există problema ca un fișier de intrare să nu fie valabil, însă impune un cost însemnat asupra scalabilității sistemului. Pe de altă parte, preprocesarea remote, abordare aleasă de **DiSCo**, micșorează durata preprocesării, însă volumul de date tranzacționat între nodurile sistemului este mai mare, deoarece nu se mai trimite prin rețea doar fișierul preprocesat, ci atât fișierul sursă, cât și toate fișierele header asociate. O abordare asemănătoare adoptă și sistemul **IncrediBuild**[18], despre care nu sunt cunoscute multe detalii de funcționare, întrucât este o soluție comercială.

În ceea ce privește arhitectura sistemului de compilare distribuită **DiSCo**, la baza acestuia se află o serie de mai multe componente:

- Command Scheduler - modul care parsează conținutul unui **Makefile** pentru a determina potențialele dependențe dintre comenzile individuale de compilare, iar în funcție de relația dintre aceste comenzi, entitățile independente sunt grupate în blocuri de comenzi, elementele unui bloc fiind executate în paralel, iar mai multe blocuri de comenzi consecutive sunt executate secvențial; totodată, modulul command scheduler folosește o metodă de planificare statică pentru a determina spre ce nod al sistemului să fie trimis un anumit bloc de comenzi
- Connection Manager - folosește un mecanism de tip thread pool pentru a deschide sau termina conexiuni cu nodurile adiacente, folosite pentru comunicarea statusului pentru un proces de compilare în desfășurare [19]

- Compilation Manager - modul responsabil pentru executarea etapelor procesului de compilare
- Network Drive Manager - gestionează transferul de fișiere în mod implicit între nodurile sistemului și mediază interacțiunea cu **H-Cache-ul** nodului, responsabil pentru stocarea eficientă de fișiere header

Figura 3.5 ilustrează arhitectura sistemului distribuit de compilare DiSCo.

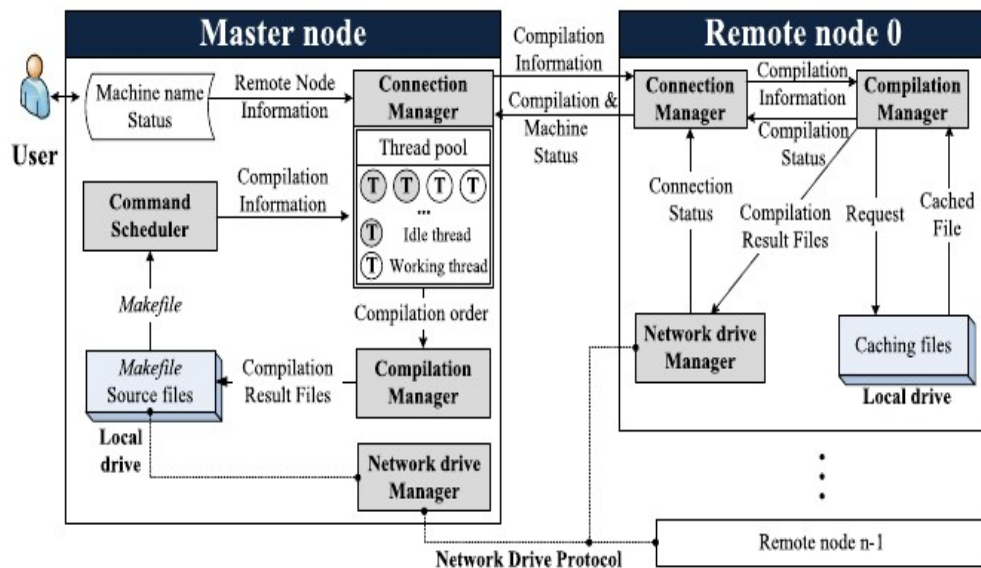


Figura 3.5: Arhitectura Disco [17]

Capitolul 4

Prezentarea proiectului

4.1 Precizări asupra conținutului și a modului de organizare

Împreună cu capitolul următor reprezintă cca. 70% din lucrare.

Titlul acestui capitol nu este unul impus și nici nu corespunde neapărat unui singur capitol. Titlul indică mai degrabă o parte (importantă și centrală, de altfel) a lucrării, în care se prezintă ceea ce s-a realizat efectiv: contribuțiile autorului. Organizarea acestei părți este dependentă și specifică fiecărei lucrări în parte și este stabilită de către fiecare autor după cum i se pare mai potrivit pentru tema lui. Ea poate cuprinde prezentarea unor concepte teoretice (unelte sau tehnici matematice folosite în lucrare, prezentarea sau introducerea unor concepte teoretice etc.), o analiză a diferitelor metode/algoritmi/tehnologii etc. luate în considerare sau dezvoltate de către autor, o prezentare a unui design (mai mult sau mai puțin detaliat) sau chiar detalii a unei eventuale implementări/prototip, dacă e cazul.

Trebuie remarcat însă faptul că această parte reprezintă contribuția personală a autorului, și în nici un caz ea nu poate fi sinteza unor texte preluate din alte surse. Prin urmare, orice informații sunt prezentate aici, ele trebuie să corespundă cel puțin unei interpretări/analize critice personale a autorului, dacă nu chiar unor idei originale ale acestuia.

4.1.1 Subsecțiune

Capitolul 5

Rezultate teoretice și experimentale

Împreună cu partea de prezentare a proiectului, trebuie să reprezinte aproximativ 70% din lucrare.

Aici sunt prezentate metodele teoretice sau practice de validare/verificare a soluțiilor propuse în partea anterioară, scenariile de testare a corectitudinii funcționale, a utilizabilității, performanței etc.

De asemenea, rezultatele testelor experimentale se pretează unor interpretări și comparații cu rezultatele altor metode similare.

Capitolul 6

Concluzii

6.1 Conținut

- un rezumat al contribuțiilor aduse
- a analiză critică a rezultatelor obținute: avantaje, dezavantaje, limitări
- o descriere a posibilelor dezvoltări și îmbunătățiri ulterioare

6.2 Detalii tehnice

6.2.1 Dimensiune

Cca 3–5% din total.

Bibliografie

- [1] J. R. Powell, “The quantum limit to Moore’s law”, *Proceedings of the IEEE*, vol. 96, nr. 8, pp. 1247–1248, 2008.
- [2] D. Kondo, B. Javadi, P. Malecot, F. Cappello și D. P. Anderson, “Cost-benefit analysis of cloud computing versus desktop grids”, în *2009 IEEE International Symposium on Parallel & Distributed Processing*, IEEE, 2009, pp. 1–12.
- [3] B. Stroustrup, *The C++ Programming Language*, 2013.
- [4] R. C. Martin, “Design principles and design patterns”, *Object Mentor*, vol. 1, nr. 34, p. 597, 2000.
- [5] D. Herity, “C++ in embedded systems: Myth and reality”, *Embedded Systems Programming*, vol. 11, nr. 2, pp. 48–71, 1998.
- [6] H. Rotithor, K. Harris și M. Davis, “Measurement and Analysis of C and C++ Performance”, *Digital Technical Journal*, vol. 10, nr. 1, pp. 32–47, 1999.
- [7] E. Burnette, *Hello, Android introducing Google’s mobile development platform: 2nd*. Android, 2009.
- [8] G. Lim, M. Lee, R. J. Lahaye și Y. I. Eom, “Distributed compilation system for high-speed software build processes”, în *2014 International Conference on Big Data and Smart Computing (BIGCOMP)*, IEEE, 2014, pp. 116–120.
- [9] M. Pool, *distcc, a fast free distributed compiler*, 2003.
- [10] W. Tarreau și D. Rodgman, *LZO stream format as understood by Linux’s LZO decompressor*, 2018.
- [11] S. Ashby, G. Eulisse, S. Schmid și L. Tuura, “Parallel compilation of cms software”, 2004.
- [12] R. Matev, “Fast distributed compilation and testing of large C++ projects”, în *EPJ Web of Conferences*, EDP Sciences, vol. 245, 2020, p. 05 001.
- [13] J. Rosdahl, *ccache - A fast compiler cache*, <https://ccache.dev/>, Accessed: 2025-02-09, 2025.
- [14] J. Rosdahl și Contributors, *ccache Manual: How ccache Works*, Accessed: 2025-02-09, 2025.

- [15] Mozilla și Contributors, *sccache - Shared Compilation Cache*, <https://github.com/mozilla/sccache>, Accessed: 2025-02-09, 2025.
- [16] T. Saario, “Enabling Sccache In CI System”, 2020.
- [17] K. Jo, S. W. Kim și J.-K. Kim, “DiSCo: Distributed scalable compilation tool for heavy compilation workload”, *IEICE TRANSACTIONS on Information and Systems*, vol. 96, nr. 3, pp. 589–600, 2013.
- [18] I. Ltd., *Incredibuild - Accelerate Development with Build Distribution*, <https://www.incredibuild.com/>, Accessed: 2025-02-09, 2025.
- [19] M. Mizuno, L. Chen și V. Wallentine, “Synchronization in a Thread-Pool Model and its Application in Parallel Computing.”, în *PDPTA*, Citeseer, 2003, pp. 1879–1885.

Anexa A

Pseudo-cod sau cod (dacă există)

```
/** Maps are easy to use in Scala. */
object Maps {
  val colors = Map("red" -> 0xFF0000,
                   "turquoise" -> 0x00FFFF,
                   "black" -> 0x000000,
                   "orange" -> 0xFF8040,
                   "brown" -> 0x804000)

  def main(args: Array[String]) {
    for (name <- args) println(
      colors.get(name) match {
        case Some(code) =>
          name + " has code: " + code
        case None =>
          "Unknown color: " + name
      }
    )
  }
}
```